

PatternBoost Three-Problem Experiment Report

Clean experiment report for the multi-level PatternBoost repository

Updated July 4, 2026

Abstract

This report summarizes the current state of the PatternBoost geometric extremal-search experiments. The active scope is restricted to three problems: maximum independent set of rectangles (MISR) LP-gap search, unit-square line-stabbing LP-gap search, and recursive guillotine hardness. The report also records a separate exploratory appendix run for `epsilon_net` and `graph_separation`. The current best audited main-track values are 1.4 for MISR, 1.5000000000000004 for unit-square stabbing, and 0.3 for guillotine hardness. The exploratory run reached 1.4545454545454546 for `epsilon_net` and no exact positive `graph_separation` witness. These values should be interpreted as the present state of a structured computational search, not as final publication-quality theorems. All reported scores must be recomputed by exact verifiers from standalone certificates before they are used in a manuscript.

Contents

1	Scope and evidence boundaries	3
2	Executive summary	4
3	Run inventory	6
3.1	Previous-best warm-start run	6
3.2	Broad 81-row run	7
3.3	Exploratory appendix run	8
4	Problem 1: MISR LP-gap search	9
4.1	Objective and exact verifier	9
4.2	Current best result	9
4.3	Strategies tested	9
4.4	Interpretation	9
4.5	Recommended next MISR changes	11
5	Problem 2: unit-square stabbing	12
5.1	Objective and exact verifier	12

5.2	Current best result	12
5.3	Strategies tested	14
5.4	Interpretation	14
5.5	Recommended next unit-square changes	14
6	Problem 3: recursive guillotine hardness	16
6.1	Objective and exact verifier	16
6.2	Current best result	16
6.3	Strategies tested	18
6.4	Interpretation	18
6.5	Recommended next guillotine changes	18
7	Why the current values are stuck	20
8	Audit rules for reporting	20
9	Recommended next experiments	21
9.1	Short term	21
9.2	Medium term	21
9.3	Publication readiness	21
10	Reproduction commands	22
A	Current matrix registry	23
A.1	MISR	23
A.2	Unit-square stabbing	23
A.3	Guillotine	23
B	Minimal certificate fields	24
B.1	MISR certificate	24
B.2	Unit-square certificate	24
B.3	Guillotine certificate	24
C	Bottom line	25

1 Scope and evidence boundaries

The main paper track currently studies three extremal-search problems:

1. **MISR LP gap:** find axis-parallel rectangle families with large LP relaxation gap for maximum independent set of rectangles.
2. **Unit-square stabbing:** find unit-square families with large integer/LP gap for stabbing by horizontal and vertical lines.
3. **Guillotine hardness:** find disjoint rectangle packings for which every recursive guillotine strategy destroys a large fraction of rectangles.

Two additional tasks, `epsilon_net` and `graph_separation`, are implemented as exploratory appendix tasks and should not be mixed with the main three-problem table. All evidence from the discarded `square-stabbing-14-9` package is excluded. In particular, the old 14/9 and 20/13 square-stabbing values should not be quoted as current evidence unless they are reimplemented and audited inside the allowed code path.

Table 1: Evidence categories used in this report.

Evidence type		Status	Interpretation
Current-code summaries	audited	Primary evidence	Safest values to cite for the current repository state.
Previous-best runs	warm-start	Primary follow-up evidence	Useful for record recovery and focused continuation, but not a neutral ablation.
Broad 81-row matrix		Diagnostic evidence	A component sweep over all three problems; useful for comparisons, but incomplete for guillotine because some rows timed out.
Exploratory run	appendix	Separate appendix evidence	Covers <code>epsilon_net</code> and <code>graph_separation</code> ; do not mix with the three-problem table.
Allowed imported square code	unit-	Allowed comparison evidence	May be used when the exact IP/LP result and certificate hash are recorded.
Discarded <code>square-stabbing-14-9</code> code		Excluded	Do not use its 14/9, 20/13, or related figures as current evidence.
Historical claims without recovered certificates		Targets only	Must be recovered and audited before citation.

The current experiment matrix has no repeated seed axis. Each row is a separate component configuration, and randomness is recorded for reproducibility. The current results are therefore best viewed as a deterministic discovery sweep and a set of focused record-recovery runs, not as a statistical estimate over repeated independent trials.

2 Executive summary

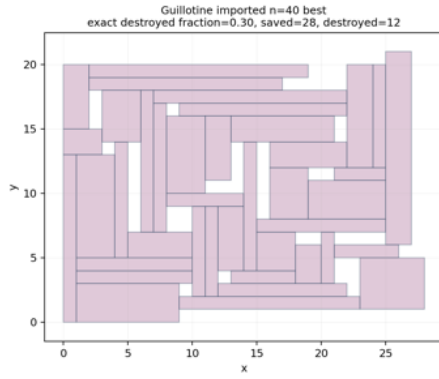
Table 2: Best current values by problem.

Problem	Best value	audited	Main source	Certificate scale and status
MISR LP gap	1.4		Previous-best warm-start	14 rectangles in the warm-start row; archived visual certificate has 34 rectangles and exact LP/ILP components.
Unit-square stabbing	1.5		Broad 81-row run and warm-start	20–23 squares in current-code rows; allowed imported artifact also records exact 3/2.
Guillotine hardness	0.3		Previous-best warm-start and exact $n = 40$ audit artifact	10 rectangles in warm-start row; exact $n = 40$ artifact destroys 12 of 40 rectangles.

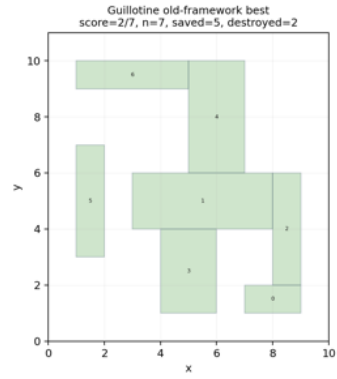
The main scientific lesson is that generic coordinate mutation is not enough. The useful progress comes when the representation and surrogate are shaped like the certificate being sought:

1. MISR improves when the search directly optimizes exact LP-gap pressure and uses sparse or triangle-free conflict structure as a prior.
2. Unit-square stabbing improves when the search works with line-square incidence and exact set-cover scoring, not only raw square coordinates.
3. Guillotine improves when the search targets recursive nonseparability and exact saved-set witnesses, not only first-cut obstruction.

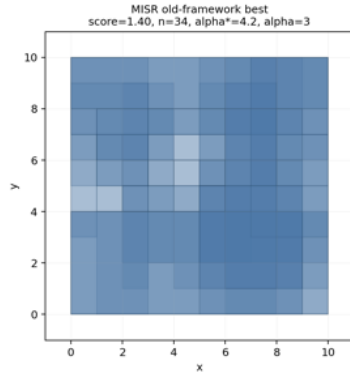
The current values are real but modest. More wall time and more epochs alone are unlikely to cross the main barriers unless the construction families and verifier feedback loops are improved.



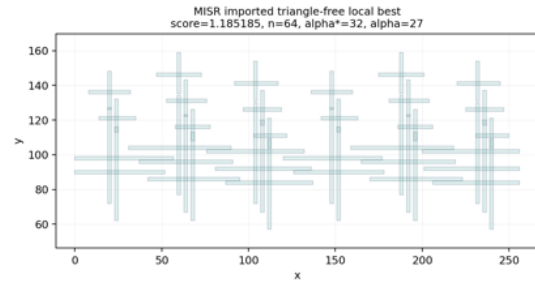
fig_guillotine_n40_0p30.png



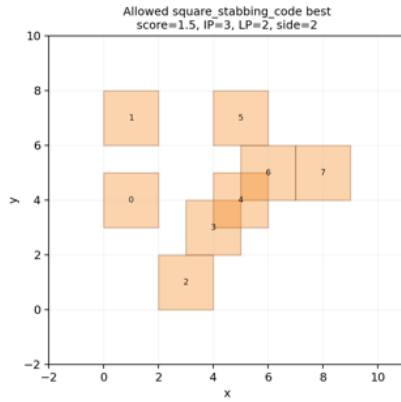
fig_guillotine_old_2_7.png



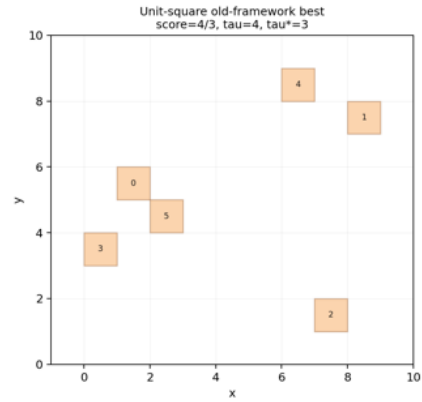
fig_misr_old_best_1p4.png



fig_misr_tf_local_1p185185.png



fig_unit_square_allowed_1p5.png



fig_unit_square_old_4_3.png

Figure 1: Contact sheet of the current curated construction figures. These renderings are diagnostic illustrations; the certificate JSON files and exact verifiers are the evidence.

3 Run inventory

3.1 Previous-best warm-start run

The most useful focused run is the previous-best warm-start run:

- Slurm job: 16493282.
- Run root: `runs/hpc_prevbest_12_4h_min8_resume_20260703_111657`.
- Status: 12 of 12 rows completed.
- Stderr: zero nonempty stderr files.
- Stop reason: all rows ended by `budget_exhausted`.
- Resume status: all rows had `resumed_from_checkpoint=True`.
- Elapsed time: about 14402–14412 seconds per row.

Table 3: Best row per problem in the previous-best warm-start run.

Problem	Score	Iter.	Configuration	Details
<code>misr</code>	1.4	7924	<code>triangle_free_rect</code> <code>lp_dual_pivot</code>	14 rectangles
<code>unit_square</code>	1.5	6687	<code>exact_lp_gap_pressure</code> <code>line_square_incidence</code> <code>coord_mutation</code>	23 squares, $\tau = 4$
<code>guillotine</code>	0.3	14556	<code>exact_stab_gap_pressure</code> <code>rect_direct_disjoint</code> <code>packing_resize</code> <code>k_subset_nonseparability</code>	10 rectangles, 3 destroyed

Table 4: All rows in the previous-best warm-start audit.

Problem	Score	Iter.	Configuration	Note
<code>misr</code>	1.4	7924	<code>triangle_free_rect</code> <code>lp_dual_pivot</code>	best MISR row
<code>misr</code>	1.375	8049	<code>exact_lp_gap_pressure</code> <code>triangle_free_rect</code> <code>lp_dual_pivot</code>	near-best
<code>misr</code>	1.375	6286	<code>triangle_free_exact_gap_pressure</code> <code>triangle_free_rect</code> <code>program_coeff_pivot</code>	near-best
<code>misr</code>	1.375	5426	<code>exact_lp_gap_pressure</code> <code>triangle_free_rect</code> <code>sequence_pair_pivot</code> <code>triangle_free_exact_gap_pressure</code>	near-best

Problem	Score	Iter.	Configuration	Note
unit_square	1.5	6687	line_square_incidence coord_mutation exact_stab_gap_pressure	size 23, $\tau = 4$
unit_square	1.5	8679	square_direct coord_mutation exact_stab_gap_pressure	size 22, $\tau = 4$
unit_square	1.428571	5118	sqstab_exact_grid coord_mutation exact_stab_gap_pressure	size 12, $\tau = 4$
unit_square	1.428571	8483	square_direct primal_dual_lines exact_stab_gap_pressure	size 30, $\tau = 4$
guillotine	0.3	14556	rect_direct_disjoint packing_resize k_subset_nonseparability	size 10, 3 destroyed
guillotine	0.25	1199	recursive_obstruction_grammar recursive_gadget_assembly depth_limited_dp	size 8, 2 destroyed
guillotine	0.25	2768	recursive_obstruction_grammar recursive_gadget_assembly first_cut_obstruction	size 8, 2 destroyed
guillotine	0.25	508	recursive_obstruction_grammar recursive_gadget_assembly k_subset_nonseparability	size 8, 2 destroyed

3.2 Broad 81-row run

The broad run tested the full no-repeated-seed component matrix:

$$3 \text{ problems} \times 3 \text{ representations} \times 3 \text{ local searches} \times 3 \text{ surrogates} = 81 \text{ rows.}$$

The run details were:

- Slurm job: 16492548.
- Run root: runs/hpc_81_4h_min8_train7_n12_venv_20260703_101409.
- Completed rows: 71 of 81.
- Timeout rows: 10, all in guillotine.
- Checkpoints: all 81 rows had checkpoints.
- Stderr: the nonempty stderr files contained Slurm time-limit messages only, not Python tracebacks.

The broad run is useful for component comparison. It should not be treated as a complete publication table until the timed-out guillotine rows are resumed or explicitly removed from the comparison. Importantly, the timed-out guillotine checkpoint values were weaker than the warm-start score of 0.3.

Table 5: Best broad-run value per problem.

Problem	Best score	Configuration	Details
<code>misr</code>	1.3333	<code>triangle_free_rect</code> <code>sequence_pair_pivot</code> <code>exact_lp_gap_pressure</code>	size 16
<code>unit_square</code>	1.5	<code>line_square_incidence</code> <code>coord_mutation</code> <code>exact_stab_gap_pressure</code>	size 20, $\tau = 4$
<code>guillotine</code>	0.1667	<code>rect_direct_disjoint</code> <code>recursive_gadget_assembly</code> <code>first_cut_obstruction</code>	size 12, destroyed 2

3.3 Exploratory appendix run

The exploratory appendix tasks were run separately from the main three-problem matrix:

- Slurm job: 16501338.
- Run root: `runs/explore_overnight_20260704_051618`.
- Preserved repository snapshot: `docs/assets/exploratory_overnight_20260704_051618/`.
- Scope: `epsilon_net` and `graph_separation`.
- Status: 12 of 12 rows completed with exit code 0:0.
- Stderr: zero nonempty stderr files.
- Matrix: 12 rows, 12 unique fresh seeds, no old fixed-seed hits.
- Validation: every best certificate verified against its summary.

Table 6: Best values in the exploratory appendix run.

Task	Best value	exact	Row	Interpretation
<code>epsilon_net</code>	1.4545454545454546		<code>eps_n11_t4_k3</code>	Nontrivial small exact witness recovered.
<code>graph_separation</code>	0.0		<code>graph_g3_n5_dense</code>	No bounded-grid mixed square/segment separation witness certified.

The best graph-separation pressure score was 1.3845054945054946 from `graph_g3_n7_motif`, but the exact score remained 0.0. This means the current stress objective can produce difficult-looking rectangle graphs, but the bounded mixed-representation verifier did not certify a separation. The next useful improvement is a stronger non-representability verifier or a more structured graph family, not simply a longer run of the same search.

4 Problem 1: MISR LP-gap search

4.1 Objective and exact verifier

For a finite family $\mathcal{R}$ of axis-parallel rectangles, the score is

$$\text{score}_{\text{MISR}}(\mathcal{R}) = \frac{\alpha^*(\mathcal{R})}{\alpha(\mathcal{R})}.$$

Here $\alpha(\mathcal{R})$ is the maximum number of pairwise-disjoint rectangles and $\alpha^*(\mathcal{R})$ is the standard clique LP optimum. The exact verifier must construct the intersection graph, enumerate the clique or point constraints needed for the LP, solve the LP and integral problem to optimality, and save the objective components.

4.2 Current best result

The best current-code warm-start row has score 1.4 with:

```
triangle_free_rect  lp_dual_pivot  exact_lp_gap_pressure.
```

The archived visual certificate shown in Figure 2 records a related old-framework MISR best with exact components:

- $n = 34$ rectangles,
- $\alpha^* = 4.2000000000000002$,
- $\alpha = 3$,
- $\alpha^*/\alpha = 1.4000000000000006$,
- certificate hash stored in `docs/assets/study_assets/misr_old_best_1p4.json`.

4.3 Strategies tested

4.4 Interpretation

The gap depends on two quantities at once: the LP value must remain high while the exact independent-set optimum remains small. The current best examples live in a small-denominator regime with α around 3, where every fractional LP improvement is amplified.

The search has not yet shown a robust way to scale the 1.4 phenomenon into a larger family. It can recover finite certificates and nearby variants, but it does not yet reliably compose them while preserving the LP/integral imbalance.

The imported triangle-free local/program track has a lower exact score 1.1851851851851851 on 64 rectangles, but it is structurally important because it enforces a sparse-conflict condition. The best use of that track is not to replace the unrestricted record search, but to split MISR into two explicit tracks: unrestricted LP-gap records and triangle-free or sparse-conflict records.

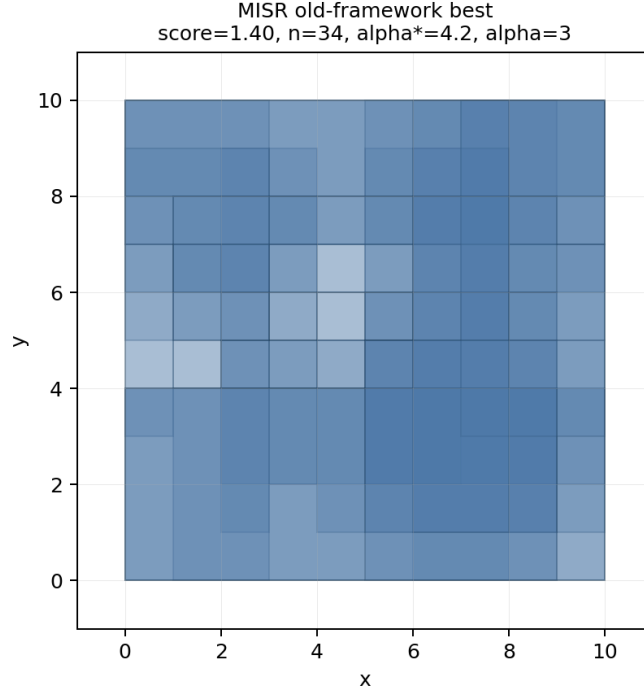


Figure 2: MISR construction associated with the archived 1.4 certificate. The exact LP/ILP certificate, not the rendering, is the proof object.

Table 7: MISR strategy summary.

Strategy	Observation
Endpoint or sequence-pair geometry	Encodes endpoint order rather than raw coordinates. This is structurally promising because swaps and block moves act on combinatorial geometry, but the current best recovered row is still tied to the <code>triangle_free_rect</code> path.
<code>triangle_free_rect</code> representation	Biases candidates toward sparse or triangle-free conflict structure. The strongest warm-start MISR rows used this representation.
<code>lp_dual_pivot</code> local search	Uses LP pressure and tight constraints to decide which rectangles to perturb. It aligns better with the objective than unguided coordinate mutation.
<code>exact_lp_gap_pressure</code> surrogate	Expensive but directly relevant. The top current MISR row uses this surrogate.
<code>graph_conflict_proxy</code> surrogate	Useful for speed, but less reliable for final quality because graph statistics can miss the LP/integral imbalance.

4.5 Recommended next MISR changes

1. Keep separate tables for unrestricted MISR and triangle-free/sparse MISR.
2. Add exact LP-dual features to triangle-free mutation; triangle-free feasibility alone is not enough.
3. Mine motifs from the 1.4 certificate and use them as optional mutation templates, while keeping neutral matrix rows randomly generated.
4. Add a composition audit: after duplicating or lifting a motif, verify whether α^* grows faster than α .
5. Recover and audit any historical 1.5 MISR example before it is cited.

5 Problem 2: unit-square stabbing

5.1 Objective and exact verifier

For a finite family $\mathcal{S}$ of unit squares, the score is

$$\text{score}_{\text{stab}}(\mathcal{S}) = \frac{\tau(\mathcal{S})}{\tau^*(\mathcal{S})}.$$

Here $\tau(\mathcal{S})$ is the minimum number of horizontal and vertical lines needed to stab all squares, and $\tau^*(\mathcal{S})$ is the LP optimum of the same set-cover formulation over the compressed critical-line universe. The verifier must save the square coordinates, critical lines, incidence matrix, integer stabbing set, LP witnesses when available, solver status, and certificate hash.

5.2 Current best result

The best current-code value is

$$1.5000000000000004.$$

The strongest rows use exact stabbing pressure:

$$\text{line_square_incidence} / \text{coord_mutation} / \text{exact_stab_gap_pressure}$$

and

$$\text{square_direct} / \text{coord_mutation} / \text{exact_stab_gap_pressure}.$$

The allowed imported unit-square artifact also records exact value $3/2$:

- exact IP value 3,
- exact LP value 2,
- gap $3/2$,
- hash 8e867b64bde38625.

For comparison, an older generic-framework example reached $4/3$ with exact components $\tau = 4$ and $\tau^* = 3$.

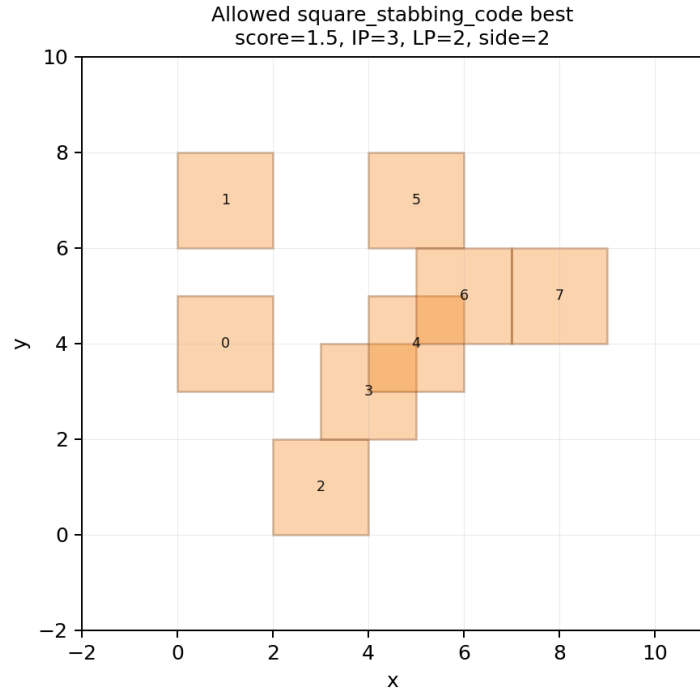


Figure 3: Allowed unit-square $3/2$ construction. This result comes from the allowed square-stabbing code path, not from the discarded `square-stabbing-14-9` package.

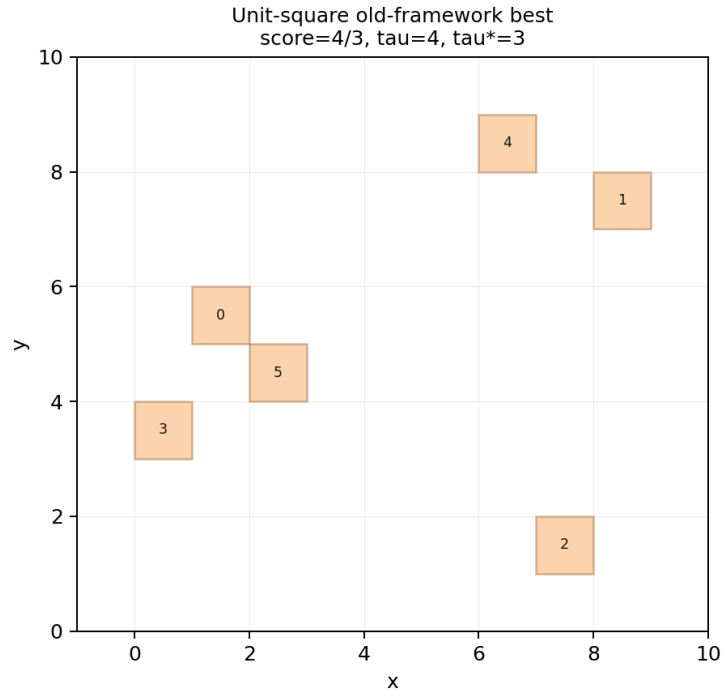


Figure 4: Old generic-framework unit-square $4/3$ example. It is useful as a diagnostic baseline but is weaker than the current allowed $3/2$ result.

5.3 Strategies tested

Table 8: Unit-square strategy summary.

Strategy	Observation
<code>square_direct</code>	Directly mutates square coordinates. It can recover 1.5 when paired with exact stabbing pressure, but it is a weak way to search for larger structured gaps.
<code>line_square_incidence</code>	Represents the incidence relation between critical lines and squares. The best broad-run unit-square row used this representation.
<code>sqstab_exact_grid</code>	Searches more directly in a grid/incidence model. It reached 1.4285714285714288 in the warm-start rows.
<code>coord_mutation</code>	Moves square positions across critical thresholds. The strongest current rows use it because exact scoring quickly filters bad perturbations.
<code>primal_dual_lines</code>	Uses line-cover information to guide local moves. It is conceptually promising but was not the top current row.
<code>exact_stab_gap_pressure</code>	Directly targets the IP/LP gap and dominates the best current unit-square rows.

5.4 Interpretation

Unit-square stabbing is essentially a set-cover integrality-gap problem with a geometric realization constraint. Raw coordinate mutation is weak because most coordinate moves do not preserve the incidence obstruction. The better code paths work because they search and score in terms of the critical line-square incidence matrix.

The current value 1.5 is meaningful, but it is also a plateau. The search is rediscovering compact $3/2$ behavior. To beat it, the allowed code path needs structured generators: staircase, ladder, cyclic threshold, or other incidence-preserving families. The discarded package should remain excluded, but its mathematical lesson is clear: stronger gaps are unlikely to appear from isolated local coordinate moves.

5.5 Recommended next unit-square changes

1. Keep `square-stabbing-14-9` excluded.
2. Reimplement only legitimate mathematical ideas inside the allowed code path: staircase-like families, ladder families, cyclic threshold patterns, and incidence-preserving generators.
3. Add richer certificate export: square coordinates, critical lines, integer stabbing set, LP primal/dual witnesses, hash, and rendering.
4. Add mutations that preserve high-value incidence structure. For example, move a square only when it remains incident to multiple active critical lines.

5. Treat 1.5 as the immediate barrier. The next goal is a verified value strictly above $3/2$ inside the allowed code path.

6 Problem 3: recursive guillotine hardness

6.1 Objective and exact verifier

For a pairwise-disjoint family $\mathcal{R}$ of axis-parallel rectangles inside a bounding box, the score is

$$\text{score}_{\text{guill}}(\mathcal{R}) = 1 - \frac{\text{save}(\mathcal{R})}{|\mathcal{R}|}.$$

Here $\text{save}(\mathcal{R})$ is the maximum number of rectangles that can be preserved by an optimal recursive guillotine-cut strategy. A cut destroys every rectangle it crosses and then recurses on the rectangles strictly on each side.

The exact verifier is a recursive dynamic program over coordinate-induced subwindows. It must save the DP value, the chosen cut tree, the number of DP states, the saved count, the destroyed count, and the exact destroyed fraction.

6.2 Current best result

The best current score is

$$0.3.$$

The warm-start row reached this value with:

`rect_direct_disjoint / packing_resize / k_subset_nonseparability.`

That row had 10 rectangles and destroyed 3.

The exact $n = 40$ audit artifact gives the same destroyed fraction at larger scale:

- $n = 40$,
- exact maximum saved: 28,
- exact minimum destroyed: 12,
- exact destroyed fraction: $12/40 = 0.30000000000000004$,
- DP states: 2478,
- target half-destruction condition: not met.

For comparison, the older small example had exact score $2/7$.

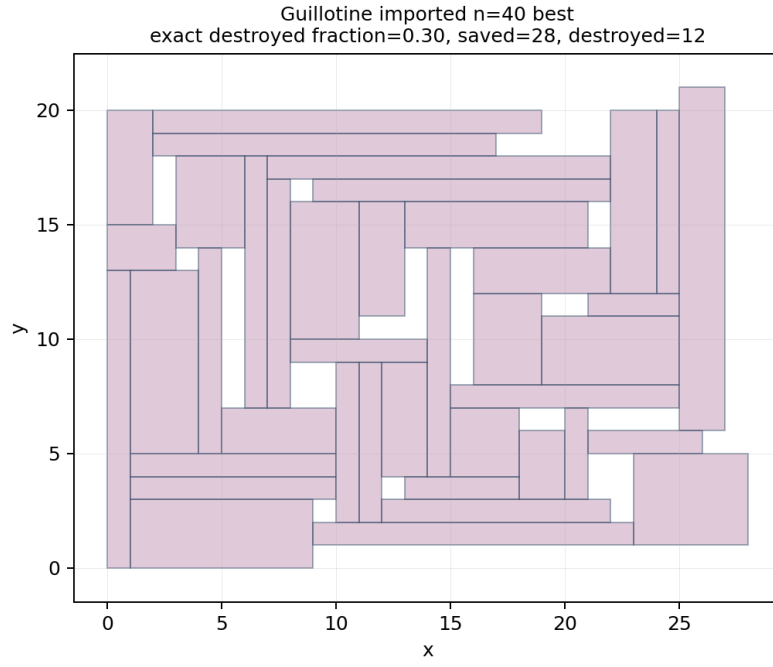


Figure 5: Exact $n = 40$ guillotine audit artifact with destroyed fraction 0.3. The sampled obstruction looked strong, but exact DP still saved 28 rectangles.

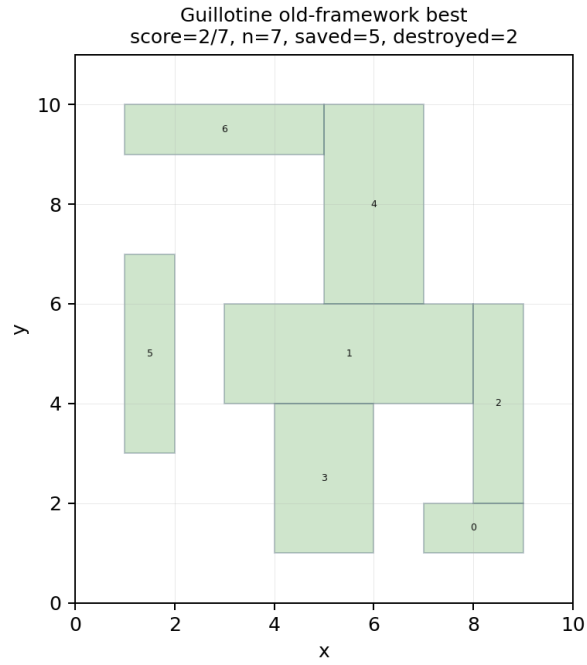


Figure 6: Older guillotine construction with exact score $2/7$.

6.3 Strategies tested

Table 9: Guillotine strategy summary.

Strategy	Observation
<code>rect_direct_disjoint</code>	Directly mutates disjoint rectangles. The best warm-start row used this representation.
<code>sequence_pair_packing</code>	Generates disjoint packings by construction. It helps feasibility, but is not currently the best row.
<code>recursive_obstruction_grammar</code>	Assembles hard subblocks and obstruction motifs. It reliably produces 0.25 examples but needs stronger witness feedback.
<code>packing_resize</code>	Moves and resizes rectangles while preserving disjointness. The best current row uses it.
<code>recursive_gadget_assembly</code>	Builds larger layouts from obstruction blocks. It creates meaningful examples, but many remain recursively separable.
<code>k_subset_nonseparability</code>	Pressures candidates by large nonseparable subsets. The best current row uses it, but sampled success is not a proof of exact recursive hardness.
<code>depth_limited_dp</code>	More aligned than first-cut-only scoring, but slower. It is useful as an elite filter.

6.4 Interpretation

Guillotine is the most deceptive of the three problems. A construction can block many first cuts and still be easy after recursion. Likewise, many random large subsets can be nonseparable while exact DP still finds a large saved subset.

The $n = 40$ artifact is the clearest example. The sampled obstruction signal was strong, but the exact audit found $\text{save} = 28$. Therefore the exact score was only $12/40 = 0.3$. The missing ingredient is not just larger layouts. The search must use the exact saved set and cut tree as feedback, then add blockers against the actual recursive escape routes.

6.5 Recommended next guillotine changes

1. Extract the exact saved set and cut tree from every high-scoring candidate.
2. Convert saved-set witnesses into mutation targets: add blockers across the exact split lines and move rectangles that enable easy recursive separation.
3. Penalize candidates by attack-found or exact $\text{save}(\mathcal{R})$, not only by sampled nonseparable subset fraction.
4. Build recursively interlocked cores so that every large subwindow inherits a hard obstruction.
5. Use exact DP earlier on medium-size candidates to avoid optimizing a misleading first-cut or sampled-subset proxy for hours.

6. Treat 0.33+ as the next practical target before attempting a half-destruction result.

7 Why the current values are stuck

The bottleneck is not simply the number of epochs. PatternBoost can train a model over the current elite distribution, but the model cannot repair a misaligned representation or a misleading surrogate. The current plateaus are:

Table 10: Current plateaus and likely causes.

Problem	Plateau	Likely cause
MISR	1.4	The search can recover finite certificates but does not yet compose them while preserving the LP/integral imbalance.
Unit-square	1.5	The search repeatedly finds compact 3/2 set-cover behavior; stronger allowed structured generators are not yet implemented.
Guillotine	0.3	Local or sampled hardness overestimates exact recursive hardness, and exact saved-set witnesses are not yet driving mutation.

The next meaningful improvements are likely to come from construction-family engineering and verifier feedback, not from simply increasing the number of random runs.

8 Audit rules for reporting

Before any value is quoted in a paper table, all of the following should hold:

1. The candidate appears in a completed `summary.json` row, not only in a checkpoint.
2. The row has `return_code = 0` or a clean and explained termination status.
3. The stop reason is recorded.
4. The certificate path exists.
5. `multilevel verify` or `multilevel audit` recomputes the same value.
6. The objective components are saved: α^*, α for MISR; τ, τ^* for unit-square stabbing; $n, \text{save}, n - \text{save}$ for guillotine.
7. Wall-time-killed rows are used only for debugging unless resumed and audited.
8. Figures are treated as illustrations, not proof objects.
9. Excluded packages remain excluded, and exploratory tasks remain separate from the main three-problem table.

9 Recommended next experiments

9.1 Short term

1. Run focused 4–8 hour follow-ups for the current best rows:
 - MISR: `triangle_free_rect` .
`lp_dual_pivot`
`exact_lp_gap_pressure`
 - Unit-square: `line_square_incidence` .
`coord_mutation`
`exact_stab_gap_pressure`
 - Guillotine: `rect_direct_disjoint` .
`packing_resize`
`k_subset_nonseparability`
2. Resume the 10 timed-out broad-run guillotine rows only if a complete component comparison is still required.
3. Add richer certificate export for unit-square and guillotine.
4. Mine motifs from current best certificates and expose them as optional mutation libraries, while keeping neutral matrix rows randomly generated.

9.2 Medium term

1. Implement allowed unit-square structured generators.
2. Add exact saved-set feedback to guillotine mutation.
3. Add LP-dual-aware triangle-free MISR mutation.
4. Build a complete per-row result table from the 81-row run after every row has clean status.

9.3 Publication readiness

A paper-ready table should include:

1. every completed row in the 81-row matrix;
2. exact score, objective components, and certificate hash;
3. time-to-best;
4. exact scorer time;
5. failure and timeout status;
6. controls;
7. either repeated runs or an explicit statement that the study is a deterministic discovery sweep rather than a repeated-run statistical study.

10 Reproduction commands

Generate the current 81-row matrix:

```
scripts/make_main_matrix.sh
```

Run one local cell:

```
PYTHONPATH=src python3 -m multilevel.cli patternboost-cell \
  --matrix runs/main_81_matrix.jsonl \
  --index 0 \
  --out-root runs/local_test \
  --iterations 20 \
  --population 16 \
  --elite 4 \
  --exact-every 5 \
  --train-every 5 \
  --model-samples 4 \
  --model-kind ngram \
  --checkpoint-every 1 \
  --n 8 \
  --grid 8
```

Audit a run:

```
PYTHONPATH=src python3 -m multilevel.cli audit \
  --root runs/local_test \
  --out runs/local_test/audit/audit.json \
  --csv runs/local_test/audit/audit.csv
```

On NYUAD Jubail, use the workflow in docs/HPC_JUBAIL.md. Always run a small smoke or slice before submitting a full array.

A Current matrix registry

A.1 MISR

```
representations:
  endpoint_sequence_pair
  triangle_free_rect
  quadratic_program_rectangles
local_search:
  sequence_pair_pivot
  lp_dual_pivot
  program_coeff_pivot
surrogates:
  exact_lp_gap_pressure
  triangle_free_exact_gap_pressure
  graph_conflict_proxy
```

A.2 Unit-square stabbing

```
representations:
  square_direct
  line_square_incidence
  sqstab_exact_grid
local_search:
  coord_mutation
  primal_dual_lines
  sqstab_local_hillclimb
surrogates:
  greedy_partial_lp_bitset
  exact_stab_gap_pressure
  incidence_statistics
```

A.3 Guillotine

```
representations:
  rect_direct_disjoint
  sequence_pair_packing
  recursive_obstruction_grammar
local_search:
  packing_resize
  recursive_gadget_assembly
  witness_breaking
surrogates:
  first_cut_obstruction
  depth_limited_dp
  k_subset_nonseparability
```

B Minimal certificate fields

B.1 MISR certificate

```
{
  "schema": "misr_certificate_v1",
  "rectangles": [[x1,x2,y1,y2], ...],
  "alpha_int": 0,
  "alpha_lp": 0.0,
  "score": 0.0,
  "maximal_cliques": [[0,3,7], ...],
  "integer_solution": [0,4,9],
  "solver_status": "optimal",
  "certificate_hash": "..."}
}
```

B.2 Unit-square certificate

```
{
  "schema": "unit_square_stab_certificate_v1",
  "squares": [[x,y], ...],
  "critical_lines": {"vertical": [x0,...], "horizontal": [y0,...]},
  "tau_int": 0,
  "tau_lp": 0.0,
  "score": 0.0,
  "integer_lines": ["v3", "h5"],
  "solver_status": "optimal",
  "certificate_hash": "..."}
}
```

B.3 Guillotine certificate

```
{
  "schema": "guillotine_certificate_v1",
  "rectangles": [[x1,x2,y1,y2], ...],
  "n": 0,
  "saved": 0,
  "destroyed": 0,
  "score": 0.0,
  "dp_states": 0,
  "optimal_strategy": {...},
  "solver_status": "optimal",
  "certificate_hash": "..."}
}
```

C Bottom line

The cleaned repository is ready for a student to run and audit the current three-problem experiments. The present main-track best values are:

Problem	Current best
MISR	1.4
Unit-square stabbing	1.5000000000000004
Guillotine	0.3

The audited exploratory appendix values are:

Task	Current best
<code>epsilon_net</code>	1.4545454545454546
<code>graph_separation</code>	0.0 exact; 1.3845054945054946 pressure

The most important next step is not a larger sweep. It is to make each search more certificate-shaped: LP-dual and sparse-conflict feedback for MISR, incidence-preserving structured generators for unit-square stabbing, and exact saved-set feedback for guillotine hardness.